

대규모 주문 트래픽 대응을 위한 이벤트 기반

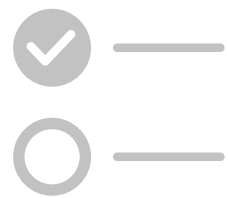
실시간 재고 처리 대시보드

●재깍 오픈(Jaekkag Open)

INDEX

목차

- ✓ 프로젝트 개요 타임세일 트래픽 폭주를 재현하는 실시간 재고 파이프라인
- ✓ 서비스 소개 주문부터 재고 차감까지, 이커머스 핵심 흐름을 3개 서비스로 설계
- ✓ 아키텍처 & 인프라 EKS·Kinesis 기반 이벤트 드리븐 구조와 비용 최적화 전략
- ✓ 핵심 구현 — NFR 검증 부하·장애·중복 상황에서도 깨지지 않는 정합성을 직접 시연
- ✓ 결과 & 회고 검증된 지표와 팀이 얻은 인사이트, 그리고 다음 단계

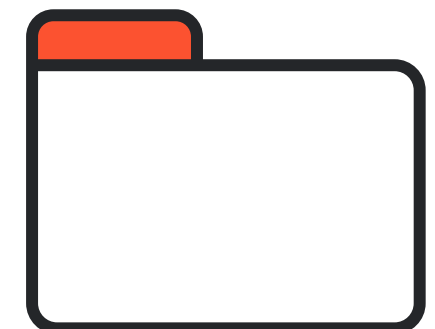


01

프로젝트 개요

● 프로젝트 배경

● 프로젝트 주제 및 목표



"갤폴드8 인기 엄청나네"...물량 부족에 사전 개통 이달 말까지 진행

등록 2026.08.05 16:46:39 | 수정 2026.08.05 17:04:24

구글에서 선호하는 매체로 추가



역대 갤럭시 최다 사전판매...일주일간 144만대 예약

폴드8 울트라 1TB 모델과 폴드8 준비 물량 초과

저장용량 업그레이드 고객 개통 기한 7일→31일



[서울=뉴스시스] 삼성전자는 지난달 28일부터 8월 3일까지 진행한 '갤럭시 Z 폴드8 울트라·폴드8·플립8' 국내 사전 판매에서 144만대를 기록하며 역대 갤럭시 스마트폰 사전 판매 신기록을 세웠다고 4일 밝혔다. 사진은 서울 서초구 삼성 강남에 전열된 '갤럭시 Z 폴드8 울트라·폴드8·플립8'. (사진=삼성전자 제공) 2026.08.04. photo@newsis.com *재판매 및 DB 금지

[서울=뉴스시스]윤현성 기자 = 역대 갤럭시 스마트폰 사전판매 신기록을 세운 삼성전자의 '갤럭시 Z8 시리즈'가 초기 흥행에 힘입어 일부 폴드 제품의 물량 부족을 빚고 있다. 삼성전자는 원하는 제품을 제때 받지 못하는 고객이 사전예약 혜택을 받을 수 있도록 개통 기간을 이달 말까지 연장했다.

삼성전자는 5일 삼성닷컴 공지를 통해 갤럭시 Z 폴드8 울트라 1테라바이트(TB) 모델과 갤럭시 Z 폴드8의 사전예약 수량이 준비된 물량을 초과했다고 밝혔다.



Jaekkak Open

프로젝트 배경

사전예약 서버는 어떻게 대규모 트래픽을 감당할 수 있을까?

예약이 시작되는 0시, 많은 사용자가 동시에 접속하면서 홈페이지에 대규모 트래픽이 발생.

저희 팀원 역시 사전예약에 참여했지만, 순간적으로 몰린 트래픽으로 인해 접속이 지연되면서 **예약에 실패.**

"수많은 사용자가 동시에 요청을 보내더라도, 서비스가 중단되지 않고 안정적으로 요청을 처리하려면 어떻게 해야 할까?"

구현 시 고려사항

이중 쓰기로 인한 데이터 정합성 문제



재고 데이터 불일치



초과 판매(Overselling)



서비스 장애 및 사용자 경험 저하



프로젝트 타겟



사용자 대상

- 판매 시작 시점에도 빠르게 주문 페이지에 접근하는 것
- 주문 버튼을 반복해서 누르지 않아도 처리 상태를 알 수 있는 것
- 주문 성공 여부가 명확한 것
- 재고가 있는데도 시스템 오류로 구매하지 못하는 상황을 줄이는 것



운영 대상

- 반복 가능한 클라우드 인프라 구성이 필요한 인프라 운영팀
- 서비스 중단 없이 애플리케이션을 배포해야 하는 개발·DevOps팀
- 주문 처리량과 재고 상태를 실시간으로 확인해야 하는 서비스 운영팀
- 트래픽 증가에 유연하게 대응할 수 있는 확장형 환경이 필요한 조직

주제

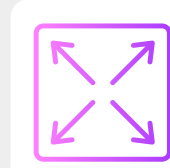


대규모 주문 트래픽 대응을 위한
이벤트 기반 실시간 재고 처리
파이프라인

- 순간 폭증하는 주문 트래픽에도 안정적으로 재고를 처리하는 파이프라인이 목표
- Outbox 패턴으로 주문 데이터와 이벤트를 원자적으로 저장하여 데이터 정합성 확보
- 4가지 핵심 원칙을 직접 구현·검증

Scalability

확장성



HPA와 Kinesis로 트래픽이 몰려도
자동으로 처리 용량을 늘립니다

Resilience

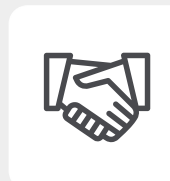
회복탄력성



EKS 기반 독립 컴포넌트 구조로 일부
장애가 전체로 번지지 않습니다

Reliability

데이터 정합성



Outbox 패턴으로 주문·재고 데이터가
유실·중복 없이 처리됩니다

Observability

관측 가능성



Prometheus·Grafana로 처리 지연과 병
목을 실시간으로 파악합니다

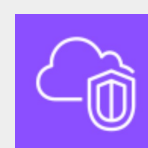
개발 환경

인프라

[클라우드]



[컴퓨팅/네트워크]



[데이터]



[보안/배포]



개발 도구

[테스트]



[코드/관리]



[AI]



[컨테이너]



협업/운영 도구

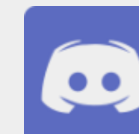
[모니터링]



[CI/CD]



[커뮤니케이션]



02

프로젝트 수행 계획 및 팀 구성

● 개발 프로세스(WBS)

● 역할 분장



02 개발 프로세스

WBS 작업 체계로 구성



업무 분장 및 개인 별 목표

반이연 | Order API

- 핵심 목표: API 확장성
- 주요 착수 포인트: 대용량 트래픽 대응 및 유연한 구조 설계

차현지 | Worker

- 핵심 목표: 재고 정합성
- 주요 착수 포인트: 동시성 제어 및 데이터 일관성 보장

김성철 | Publisher

- 핵심 목표: 이벤트 안정성
- 주요 착수 포인트: 메시지 유실 없는 안정적인 이벤트 발행

이광훈 | Infra / CI-CD

- 핵심 목표: 자동화 운영
- 주요 착수 포인트: 무중단 배포 환경 구축 및 파이프라인 자동화

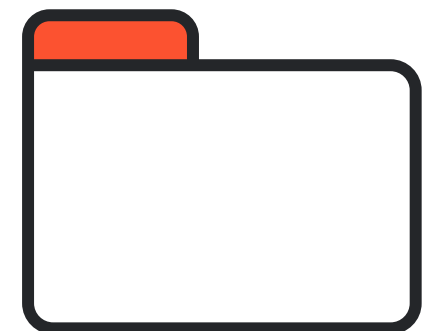


03

서비스 소개

● 서비스 개요

● 서비스 플로우

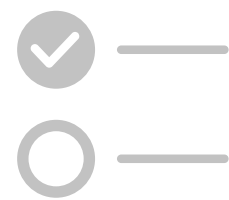


03

서비스 개요

● 재깍 오픈(Jaekkag Open)

신형 스마트폰 사전예약 오픈 시 집중되는 주문 요청을
자동 확장으로 수용하고,
주문 접수와 재고 처리를 비동기로 분리해
모델·색상별 한정 재고를 안정적으로 관리합니다.



오전 10시 사전 예약 시작



모델, 색상별 한정 수량 판매



예약 시작과 동시에 주문 요청 집중



동일한 마지막 재고에 여러 주문이 동시 접근

03 재깍오픈 서비스 구성



역할 분리(Role)

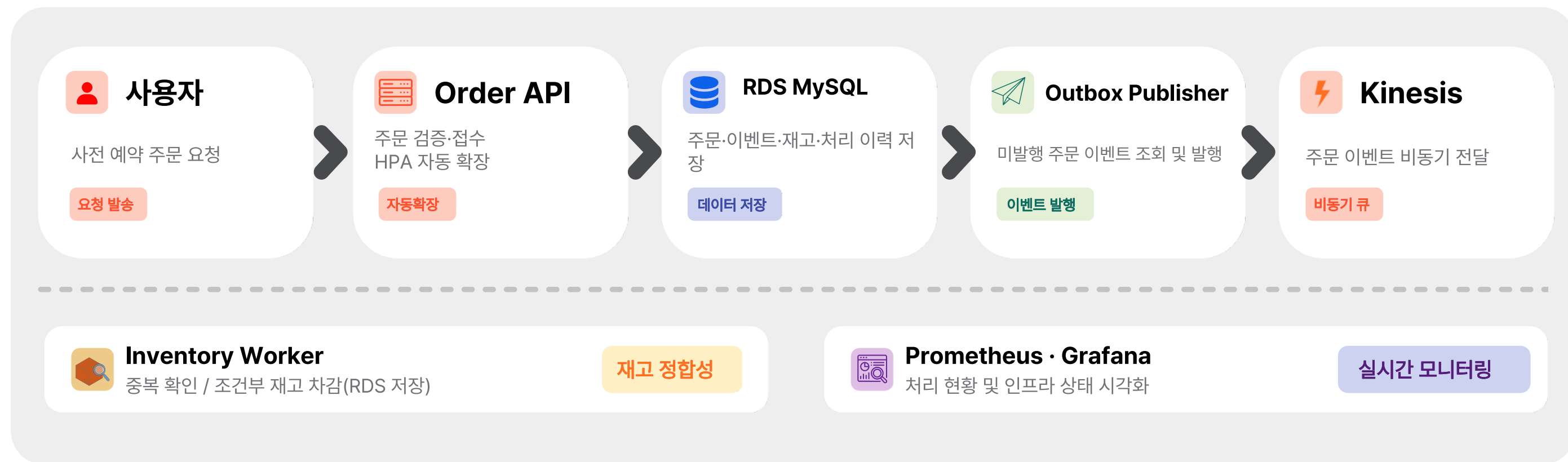
API 접수와 Worker 재고 차감의 명확한 독립



장애 관측 (Metric)

지연·오류 및 재고 상태 실시간 관측

주문 접수부터 재고 처리까지 역할을 분리하여, 트래픽 급증과 한정 재고 동시 주문에 대응합니다.



● 자동확장(Order API/ HPA)

● 비동기 처리(Outbox & Kinesis)

● 재고 정합성(Inventory Worker)

● 모니터링 (Prometheus & Grafana)

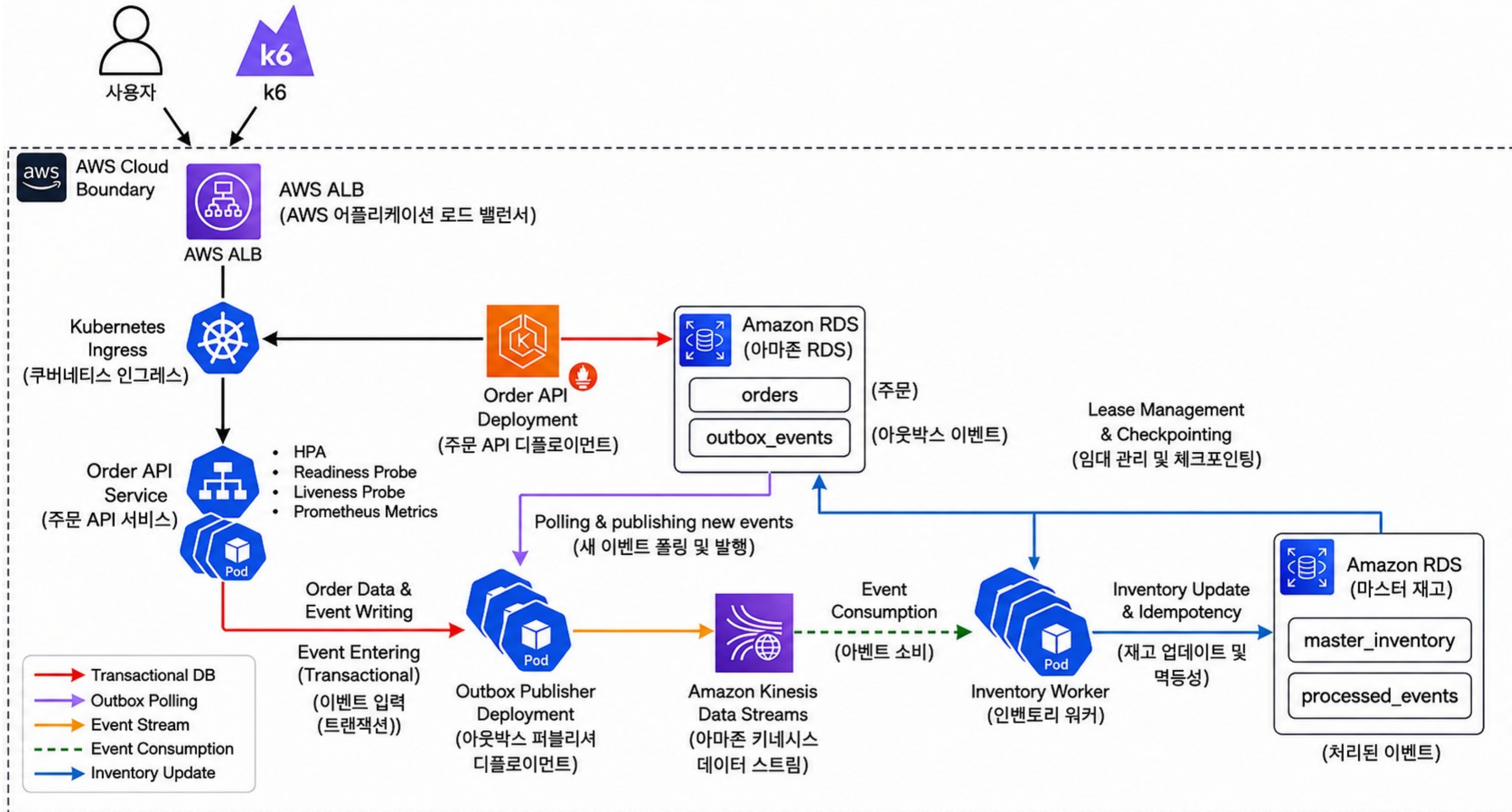
04

아키텍처 & 인프라

- 아키텍처 구성 + CI/CD 파이프라인 구조

- AWS 클라우드 비용 절감 전략

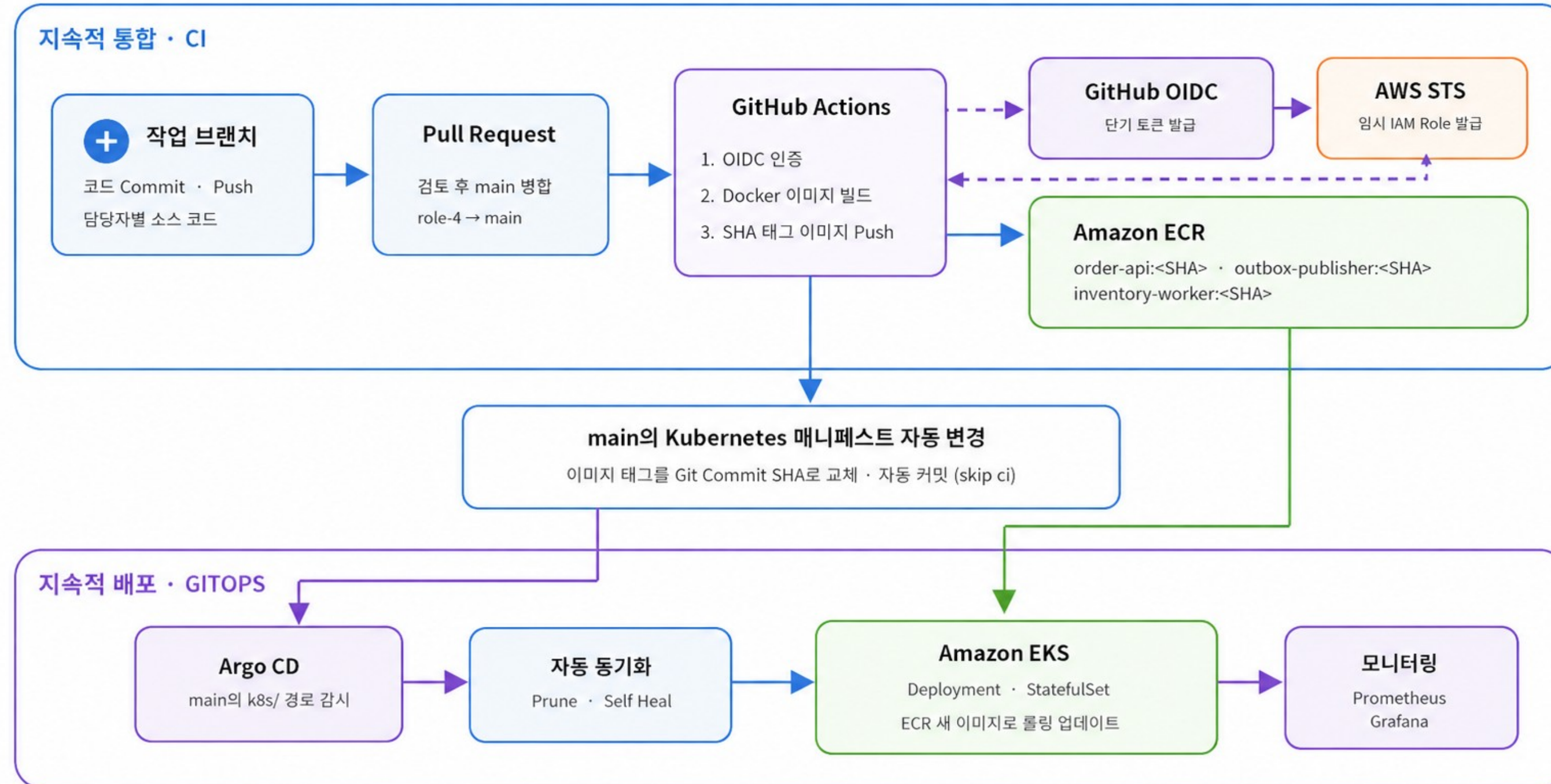




JAEKKAG CI/CD 및 GitOps 파이프라인

GitHub Actions CI · Amazon ECR · Argo CD · Amazon EKS

지속적 통합 · CI



CI: 이미지 빌드 · Push · CD: Git 기준 자동 배포 · 인증: 장기 AWS Key 없는 GitHub OIDC

AWS 클라우드 비용 절감 전략

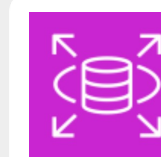
3 Shard



Kinesis PROVISIONED 모드

On-Demand는 처리량 단가가 높아
예상 트래픽 기준 샤드 수 사전 산정

-50%



RDS Multi-AZ 비활성화

Dev 환경은 상시고가용성 불필요, 단일
AZ로 인스턴스 비용 절반 절감

최근 10개



ECR Lifecycle Policy

이미지 스토리지 \$0.10/GB·월
최근 10개만 유지해 누적 스토리지 비용 절감

On/Off

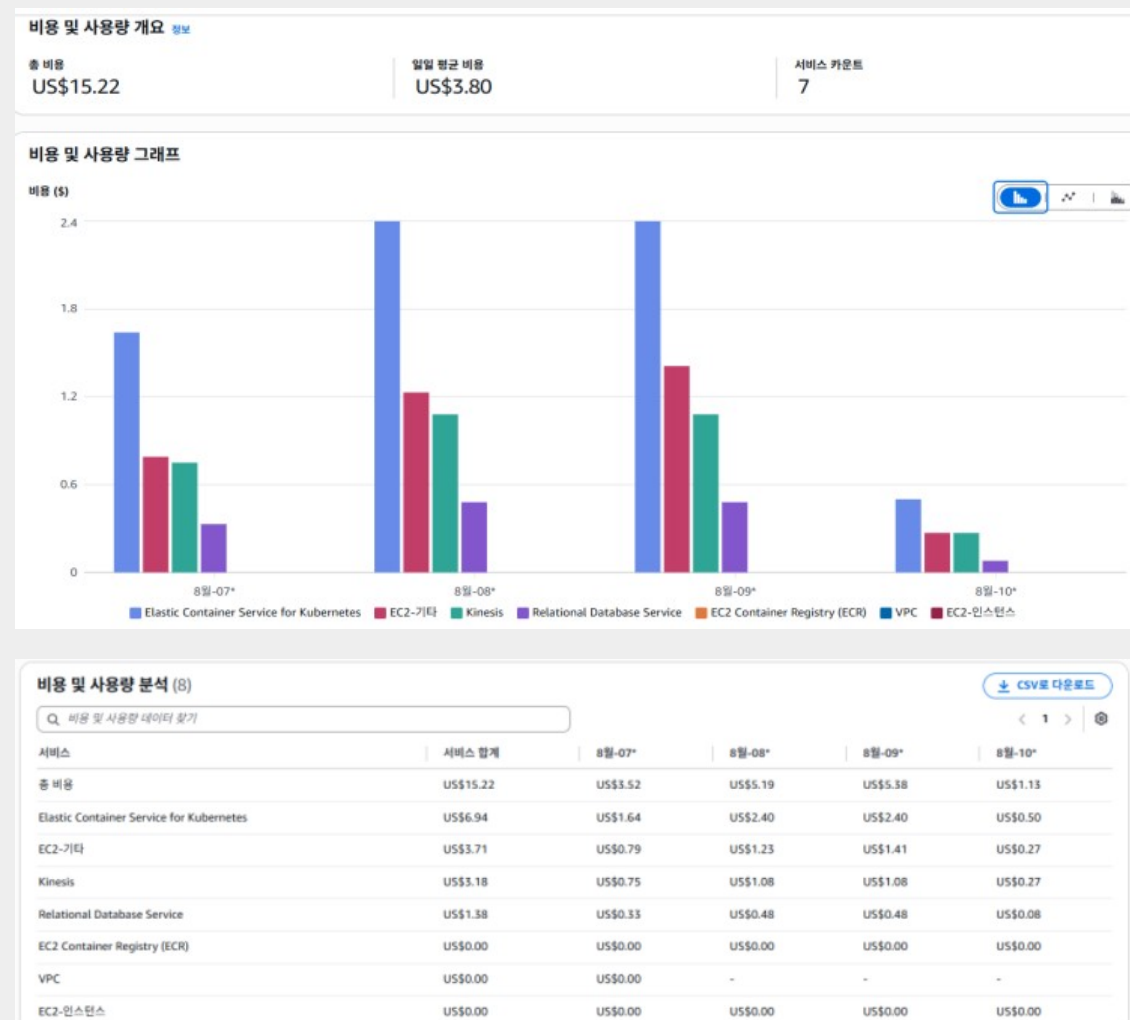


Terraform 기반 IaC 운영

작업 없는 시간엔 destroy,
재개 시 apply로 유휴 인프라 비용 제거

AWS 클라우드 리소스 비용/ 사용량 보고서

실제 사용량



예상 비용

8월 8일 -10일, 총 72시간 운영시 예상 비용 약 \$25-35

리소스	구성	3일 예상
EKS Control Plane	1개 × \$0.10/h	\$7.20
EC2 Worker	t3.medium 2~3대	\$6.00~9.00
EBS	gp3 20GB × 2~3	\$0.32~0.48
RDS	db.t3.small, Single-AZ	약 \$2.45
RDS Storage	gp3 20GB	약 \$0.23
NAT Gateway	1개	\$3.24
Kinesis	3 Shards	\$3.24
ALB	1개	약 \$1.62+
Public IPv4	NAT-ALB 주소	약 \$1.08
ECR/CloudWatch	사용량 기반	소액

04

핵심 구현 — NFR 검증

● 테스트 시나리오

● 데모 시연



테스트 시나리오

STEP 1 · 준비

평상시 흐름

정상 주문으로 기본 파이프라인 동작 확인

STEP 2 · 확장성

스파이크 부하

트래픽 급증 → HPA 확장, Kinesis lag 관찰

STEP 3 · 회복탄력성

Pod 강제 장애

Pod 삭제 → 자동 재생성, 이벤트 유실 없음

STEP 4 · 데이터 정합성

재고 소진 테스트

재고 15개 세팅 → 성공 15건, 이후 OUT_OF_STOCK, 음수 없음

STEP 5 · 데이터 정합성

중복 이벤트 발행

동일 event_id 2회 발행 → 1건만 처리 (멱등성 확인)

STEP 6 · 관측 가능성

최종 정합성 확인

Outbox 적체 0건, 재고 음수 0건, 지표로 확인

STEP 1 · 준비

[테스트 결과]

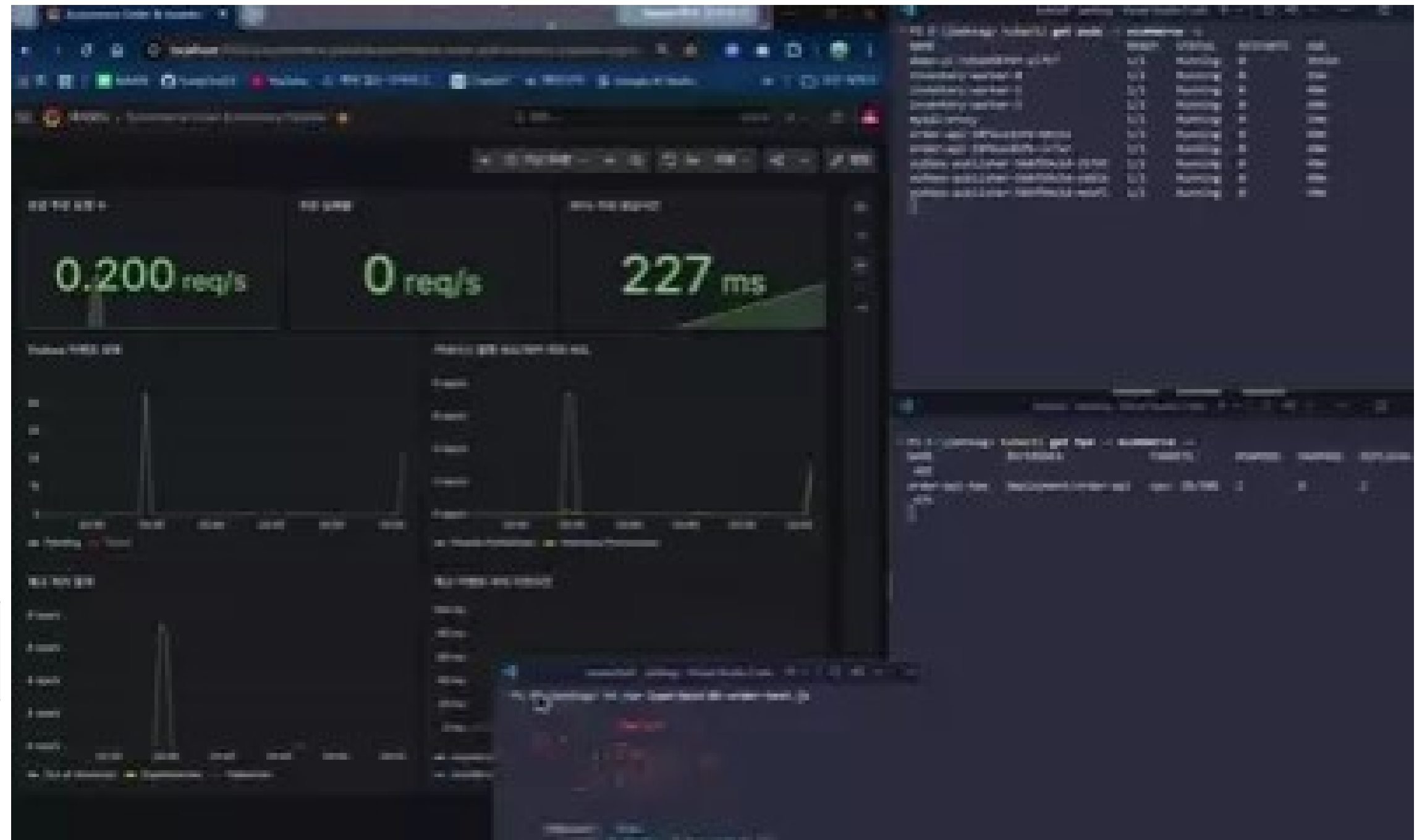
성공률: 100% (902/902 체크 통과,
0건 실패)

총 요청: 451건 (20초 동안)

초당 처리: 22.35건/초

평균 응답시간: 219ms

부하 증가에 따라 HPA가 Order API를 2개에서 3개로 확장



STEP 2 · 확장성

[테스트 결과]

성공률: 100% (3,314/3,314 체크 통과,
0건 실패)

총 요청: 3,314건 (55초 동안, 초당 최대
90건까지 상승)

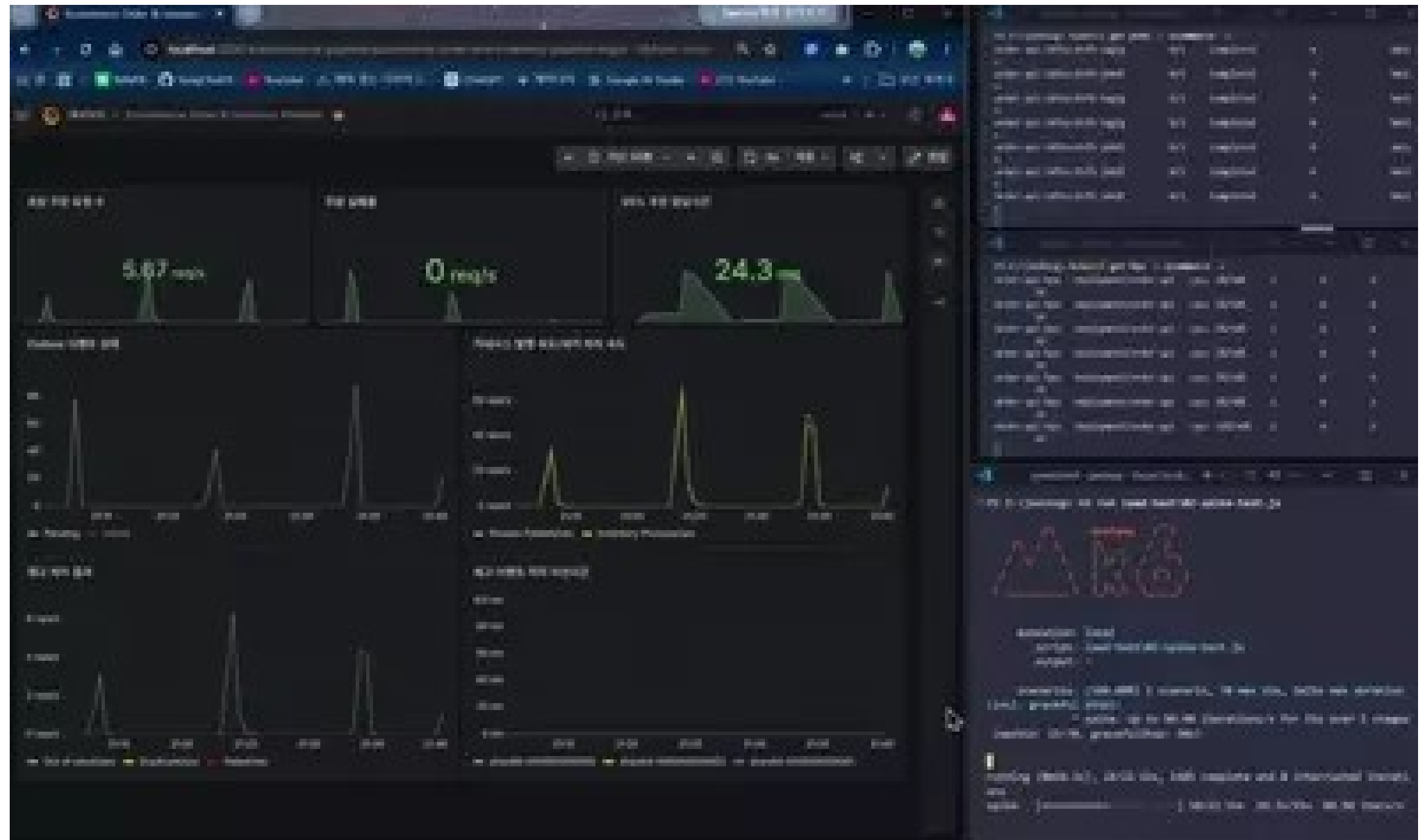
초당 처리: 60.25건/초

평균 응답시간: 246.48ms

p(95) 응답시간: 403.77ms

최대 응답시간: 977.89ms

HPA가 Order API Pod 2 → 4 자동 확장



STEP 3 · 회복탄력성

Inventory Worker Pod를 강제로 삭제한 뒤, Kubernetes가 새로운 Pod를 자동 재생성하고 이벤트 처리를 계속하는지 확인

```
NAME                READY   STATUS    RESTARTS   AGE
inventory-worker-0   1/1     Running   0           90s
inventory-worker-1   1/1     Running   0           11m
inventory-worker-2   1/1     Running   0           12m
inventory-worker-0   1/1     Terminating 0           2m14s
inventory-worker-0   1/1     Terminating 0           2m14s
inventory-worker-0   0/1     Error      0           2m14s
inventory-worker-0   0/1     Error      0           2m15s
inventory-worker-0   0/1     Error      0           2m15s
```

STEP 4 · 데이터 정합성

한정 재고 15개에 동시 주문 발생
→ 15건만 정상 차감되고 이후 주문은
OUT_OF_STOCK 처리되는지 검증

The screenshot displays a development environment with two main components:

PowerShell Script (02-spike-test.ps1):

```
execution: local
script: load-test\05-stockout-test.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 40s max duration (incl. graceful stop):
  * stockout: 10 looping VUs for 10s (gracefulStop: 30s)
```

SQL Query (Query 1):

```
1 select * from master_inventory
2 where product_id = 104
```

Database Schema (ecommerce):

- Tables
 - master_inventory
 - orders
 - outbox_events
 - processed_events
- Views
- Stored Procedures
- Functions

Result Grid:

product_id	model_name	color_name	stock_quantity	updated_at
104	FOLD	GRAY	15	2026-08-10 11:25:34.752845

STEP 5 · 데이터 정합성

동일한 event_id를 2회 발행 →
최초 1건만 처리되고 중복 이벤
트는 무시되는지 검증

The screenshot shows a PowerShell terminal at the top and SQL Server Enterprise Manager at the bottom. The PowerShell terminal displays a REST client request to a REST API, sending a POST payload with an event_id and product_id. The SQL Server Enterprise Manager interface shows a query window with a SQL query to select from the master_inventory table where product_id is 101. Below the query window, the Results pane shows a single row of data. At the bottom, the Activity Monitor pane shows a list of recent queries, including the one executed in the query window.

```
PowerShell Terminal Output:  
PS C:\Jaekak> .\load-test\api\create-test.ps1  
event_id: 82ad9561-9b8b-4bfa-9c18-f59a3af03896  
[{"event_type": "ORDER_CREATED", "created_at": "2024-08-16T11:45:53Z", "quantity": 1, "event_id": "82ad9561-9b8b-4bfa-9c18-f59a3af03896", "o  
rder_id": "99999", "product_id": "101"}]  
  
SQL Server Enterprise Manager Query Window:  
1 select * from master_inventory  
2 where product_id = 101;  
3  
  
Results Table:  
product_id | model_name | color_name | stock_quantity | updated_at  
-----  
101 | FOLD | BLACK | 20 | 2024-08-16 11:45:53.985Z  
9999 | 9999 | 9999 | 9999 | 9999  
  
Activity Monitor Log:  
# | Time | Action | Message | Duration / Rows  
1 | 20:41:26 | select "from orders LIMIT 0, 1000" | Error: Code: 1040. No database selected. Select the default DB to be... | 0.203 sec  
2 | 20:41:38 | use master; | 0 rows affected | 0.203 sec  
3 | 20:41:58 | use master_inventory; | Error: Code: 1040. Unknown database 'master_inventory' | 0.203 sec  
4 | 20:42:30 | select "from master_inventory LIMIT 0, 1000" | 12 rows returned | 0.204 sec / 0.200 sec
```

STEP 6 · 관측 가능성

Outbox 적체 확인 (PENDING/FAILED가 0이어야 함)

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	process_status	COUNT(*)			
▶	OUT_OF_STOCK	1042			
	SUCCESS	10596			

재고 음수 확인 (결과가 아예 안 나와야 정상)

Result Grid						Filter Rows:	Edit:
	product_id	model_name	color_name	stock_quantity	updated_at		
*	NULL	NULL	NULL	NULL	NULL		

05

결과 & 회고

● 트러블 슈팅 및 개선

● 프로젝트 결과

● 향후 계획

● 팀 회고



⚠ 트러블 슈팅 1: 부하 증가에 따른 Pod/Node 확장 실패

증상

Spike Test 중 HPA가 Order API Pod 확장을 시도했지만 추가 Pod가 Pending 상태로 머무름

Spike Test

k6 요청 증가
Order API CPU 상승



HPA Scale-out

k6 요청 증가
replicas 2 → 4 시도



Pod Pending

FailedScheduling
Too many pods



Service Risk

기존 Pod에 부하 집중
Probe timeout 가능

원인

- Pod가 특정 Node에 편중
- Node의 Pod 수용 한도 도달
- HPA는 Pod만 늘리고 Node는 직접 늘리지 못함

개선

- topologySpreadConstraints 적용
→ Order API Pod를 Node 간 분산
- **Cluster Autoscaler 적용 (노드그룹 단위)**
→ Pending Pod 발생 시 Node 자동 확장



트러블 슈팅 2: 인프라 확장 이후 DB / Connection 병목 발생

관측

Node 확장 후에도 /orders 500 에러가 반복 → DB 연결 경로를 추가 분석

병목 이동 흐름

Node 병목 해소

topology spread
Cluster Autoscaler

RDS 연결 압박

max_connections 61
최대 사용 54

QueuePool 고갈

size 10 + overflow 15
30초 timeout

실제 로그

```
QueuePool limit of size 10  
overflow 15 reached,  
connection timed out,  
timeout 30.00
```

POST /orders → 500

원인

Order API 내부 SQLAlchemy Pool에
서 연결을 빌리지 못해 timeout 발생

개선

- RDS 스케일업 진행 (micro → small)
- Order API Pool 조정
→ pool_size / max_overflow 재검토

프로젝트 결과

01. 확장 가능한 주문 처리 인프라 구현



k6 부하 테스트와 HPA를 연계하여 주문 폭주 시 Pod가 자동 확장되는 구조를 구현하고 검증했습니다.

02. 이벤트 기반 비동기 재고 처리 구현



Outbox Pattern과 Kinesis를 활용해 주문 API와 재고 처리를 분리하고, 대량 주문 발생 시에도 안정적으로 이벤트를 처리할 수 있는 구조를 구축했습니다.

03. 실제 부하 테스트를 통한 병목 발견 및 개선



Pod 쏠림, Pod 수용 한도 초과, Probe 실패 등 실제 운영 환경에서 발생할 수 있는 문제를 발견하고 스케줄링 및 인프라 확장 관점에서 개선했습니다.



향후 계획



STEP 01

Outbox 발행 방식 개선 (CDC 전환)

DB 폴링 방식 → CDC
실시간 변경 감지로 전환

STEP 02

자동 복구 시스템

재고 불일치 자동 검증
실패 이벤트 자동 재처리

STEP 03

Grafana Alerting → Slack / Amazon SNS 연동

장애 발생 시 실시간 알림

STEP 04

LLM 기반 관리자 Dashboard

자연어 질의
"어디 물량이 제일 많이 팔렸어?"
"인기 없는 매물은 뭐야?"

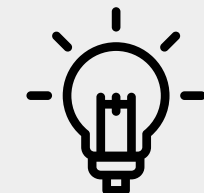
팀 회고

잘한 점



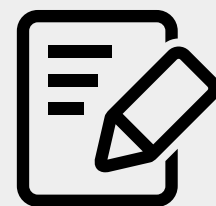
역할 분담, 이벤트 기반 아키텍처 구현, 반복 부하 테스트, 모니터링 기반 개선

어려웠던 점



짧은 개발 기간, Node·DB 병목, 중복 처리 검증, 팀 작업 통합

배운 점



전체 시스템 관점의 확장, Outbox와 멍등성, 관측 가능성의 중요성

개선할 점



사전 규칙 문서화, 자동화 테스트, Connection 최적화, 장애 대응 체계화



Thank You

감사합니다

“정해진 시간에 재깍 열리고, 폭주하는 주문도 재깍 처리한다.”